

Git

1. Overview

Git is a distributed version control system used to track changes in files, coordinate work between developers, and safely manage different versions of a project.

Definition

Git records snapshots of a project over time. Each snapshot is called a commit. Developers can move between commits, compare changes, create branches, merge work, undo mistakes, and collaborate through remote repositories such as GitHub, GitLab, or Bitbucket.

Why It Matters

Git matters because real software projects change constantly. Multiple developers may edit the same codebase, bugs need to be fixed without breaking production, features need to be developed separately, and teams need a reliable history of what changed, when, and why.

Where It Is Used in Real Systems

Git is used in almost every modern software workflow:

- Backend services and APIs
- Frontend web applications
- Mobile apps
- Database migration repositories
- DevOps and infrastructure-as-code projects
- Open-source projects
- CI/CD pipelines
- Code reviews and pull requests

Why Interviewers Ask About It

Interviewers ask Git questions because Git is part of daily engineering work. They want to know whether you can:

- Work safely in a team codebase
- Understand branch-based workflows
- Resolve merge conflicts
- Undo mistakes without damaging shared history
- Use GitHub pull requests and reviews
- Explain the difference between commands like `merge`, `rebase`, `reset`, and `revert`

2. Core Idea

The core idea of Git is that your project history is a chain, or graph, of commits. Each commit stores a snapshot of the project and points to its parent commit.

Intuition

Think of Git as a time machine for your code. Every time you commit, you save a meaningful checkpoint. If something breaks later, you can inspect, compare, or return to an earlier checkpoint.

Real-World Analogy

Imagine writing a long assignment with multiple versions:

```
assignment-v1.docx
assignment-v2.docx
assignment-final.docx
assignment-final-fixed.docx
assignment-final-real-final.docx
```

Git replaces this messy system with structured history:

```
A -- B -- C -- D
```

Each letter is a commit. Git knows exactly what changed between versions.

Small Example

```
git init
git add app.py
git commit -m "Add initial app"
git branch feature-login
git switch feature-login
git commit -am "Add login validation"
git switch main
git merge feature-login
```

This creates a repository, commits a file, creates a branch, works on that branch, and merges it back into `main`.

Step-by-Step Explanation

1. You edit files in the working directory.
2. You choose files to include using `git add`.
3. Git places those changes in the staging area.
4. You create a commit using `git commit`.
5. The commit becomes part of local history.
6. You share commits with a remote using `git push`.
7. You receive others' commits using `git pull` or `git fetch`.

```
Working Directory -> Staging Area -> Local Repository -> Remote Repository
    edit              git add              git commit              git push
```

3. Important Subtopics

3.1 Repository

What It Means

A repository, or repo, is a project folder tracked by Git. It contains your files and a hidden `.git` directory that stores history and metadata.

Why It Matters

The `.git` directory is what makes a normal folder into a Git repository.

Example

```
git init
```

This creates a new Git repository in the current folder.

Common Interview Angle

Interviewers may ask: "What happens when you run `git init`?"

Expected answer: It creates a `.git` folder containing Git's internal database, references, configuration, and object storage.

3.2 Clone

What It Means

`git clone` copies a remote repository to your local machine.

Why It Matters

It is the usual first step when joining an existing project.

Example

```
git clone https://github.com/user/project.git
```

Common Interview Angle

Interviewers may ask the difference between downloading a ZIP and cloning.

Key point: Cloning includes full Git history and remote tracking information. A ZIP usually contains only the current files.

3.3 Add

What It Means

`git add` moves changes from the working directory into the staging area.

Why It Matters

It lets you choose exactly what should go into the next commit.

Example

```
git add file.txt
git add .
git add -p
```

`git add -p` lets you stage selected parts of a file.

Common Interview Angle

Interviewers may ask why Git has a staging area.

Expected answer: The staging area helps create clean, logical commits instead of committing every local change together.

3.4 Commit

What It Means

`git commit` creates a permanent snapshot of staged changes in local history.

Why It Matters

Commits are the main units of project history.

Example

```
git commit -m "Fix user login validation"
```

Common Interview Angle

Interviewers may ask what makes a good commit.

Good commits are small, focused, buildable, and have meaningful messages.

3.5 Push

What It Means

`git push` uploads local commits to a remote repository.

Why It Matters

It shares your work with teammates and CI/CD systems.

Example

```
git push origin main
git push -u origin feature-login
```

Common Interview Angle

Interviewers may ask why a push gets rejected.

Common reason: The remote branch has commits you do not have locally. You need to pull or rebase first.

3.6 Pull

What It Means

`git pull` downloads remote changes and integrates them into your current branch.

Internally:

```
git pull = git fetch + git merge
```

or, if configured:

```
git pull --rebase = git fetch + git rebase
```

Why It Matters

It keeps your local branch updated with remote work.

Example

```
git pull origin main  
git pull --rebase origin main
```

Common Interview Angle

Interviewers may ask the difference between **fetch** and **pull**.

Key point: **fetch** only downloads data. **pull** downloads and integrates it.

3.7 Branch

What It Means

A branch is a movable pointer to a commit. It lets you develop work separately from other work.

Why It Matters

Branches allow parallel development without disturbing stable code.

Example

```
git branch feature-login  
git switch feature-login  
git switch -c feature-payment
```

Common Interview Angle

Interviewers may ask: "Is a Git branch a copy of the full project?"

Answer: No. A branch is a lightweight pointer to a commit.

3.8 Merge

What It Means

git merge combines changes from one branch into another.

Why It Matters

It is commonly used to bring feature branch work into **main**.

Example

```
git switch main
git merge feature-login
```

Common Interview Angle

Interviewers may ask what a merge commit is.

A merge commit is a commit with two or more parent commits, created when Git combines divergent histories.

3.9 Rebase

What It Means

`git rebase` moves or reapplies commits from one branch on top of another commit.

Why It Matters

It creates a cleaner, linear history.

Example

```
git switch feature-login
git rebase main
```

Before:

```
main:    A -- B -- C
feature:  \-- D -- E
```

After rebasing feature onto main:

```
main:    A -- B -- C
feature:  \-- D' -- E'
```

D' and **E'** are new commits because rebase rewrites commit history.

Common Interview Angle

Interviewers may ask: "Why should you avoid rebasing public branches?"

Because rebase rewrites history. If others already based work on the old commits, rewriting them can create confusion and conflicts.

3.10 Stash

What It Means

`git stash` temporarily saves uncommitted changes and returns your working directory to a clean state.

Why It Matters

It helps when you need to quickly switch branches or pull changes without committing incomplete work.

Example

```
git stash
git stash list
git stash pop
git stash apply
```

Common Interview Angle

Interviewers may ask the difference between `stash pop` and `stash apply`.

`pop` applies the stash and removes it from the stash list. `apply` applies it but keeps it saved.

3.11 Reset

What It Means

`git reset` moves the current branch pointer to another commit. Depending on the mode, it also changes the staging area and working directory.

Why It Matters

It is powerful for undoing local changes, but dangerous on shared branches.

Example

```
git reset --soft HEAD~1
git reset --mixed HEAD~1
git reset --hard HEAD~1
```

Common Interview Angle

Interviewers often ask the difference between soft, mixed, and hard reset.

Command	Moves HEAD	Changes Staging Area	Changes Working Directory	Use Case
<code>git reset --soft</code>	Yes	No	No	Undo commit, keep changes staged
<code>git reset --mixed</code>	Yes	Yes	No	Undo commit, keep changes unstaged
<code>git reset --hard</code>	Yes	Yes	Yes	Delete commits and local changes

3.12 Revert

What It Means

`git revert` creates a new commit that undoes the changes introduced by a previous commit.

Why It Matters

It is safer than reset for shared branches because it does not rewrite history.

Example

```
git revert abc123
```

Common Interview Angle

Interviewers may ask when to use revert instead of reset.

Use revert on public/shared history. Use reset mainly for local, unpublished cleanup.

3.13 Cherry-Pick

What It Means

`git cherry-pick` applies a specific commit from one branch onto another branch.

Why It Matters

It is useful when you need one fix from another branch without merging the whole branch.

Example

```
git cherry-pick abc123
```

Common Interview Angle

Interviewers may ask for a real use case.

Example: A bug fix was made on `develop`, and the same fix is urgently needed on `release`.

3.14 Conflict Resolution

What It Means

A conflict happens when Git cannot automatically combine changes from different branches.

Why It Matters

Conflict resolution is a normal part of team development.

Example Conflict

```
<<<<<< HEAD
return "Hello from main";
=====
return "Hello from feature";
>>>>>> feature-branch
```

You manually choose or combine the correct code:

```
return "Hello from feature";
```

Then:

```
git add file.txt
git commit
```

During rebase:

```
git add file.txt
git rebase --continue
```

Common Interview Angle

Interviewers may ask: "How do you resolve a merge conflict?"

Expected answer:

1. Run `git status`.
2. Open conflicted files.
3. Find conflict markers.
4. Decide correct final code.
5. Remove markers.
6. Test the code.
7. Stage resolved files.
8. Continue merge or rebase.

3.15 Fetch

What It Means

`git fetch` downloads commits, branches, and tags from a remote without changing your working branch.

Why It Matters

It lets you inspect remote changes before integrating them.

Example

```
git fetch origin
git log HEAD..origin/main
```

Common Interview Angle

`fetch` is safer than `pull` when you want to inspect first.

3.16 Status

What It Means

`git status` shows the current state of your working directory, staging area, and branch.

Why It Matters

It is the first command to run when you are unsure what is happening.

Example

```
git status
```

Common Interview Angle

Interviewers may expect you to use `git status` while resolving conflicts or undoing mistakes.

3.17 Log

What It Means

`git log` shows commit history.

Why It Matters

It helps inspect what changed and who changed it.

Example

```
git log
git log --oneline --graph --decorate --all
```

Common Interview Angle

Knowing history helps debug regressions and understand branch structure.

3.18 Diff

What It Means

`git diff` shows line-by-line differences.

Why It Matters

It helps review changes before staging, committing, or pushing.

Example

```
git diff
git diff --staged
git diff main..feature-login
```

Common Interview Angle

Interviewers may ask how you inspect what you are about to commit.

Answer: Use `git diff` and `git diff --staged`.

3.19 Restore

What It Means

`git restore` discards or restores file changes.

Why It Matters

It is a safer, clearer modern command for file-level undo operations.

Example

```
git restore file.txt  
git restore --staged file.txt
```

Common Interview Angle

`git restore --staged` unstages a file without deleting the file changes.

3.20 Tag

What It Means

A tag is a named pointer to a specific commit, often used for releases.

Why It Matters

Production releases are commonly marked using tags.

Example

```
git tag v1.0.0  
git push origin v1.0.0
```

Common Interview Angle

Interviewers may ask the difference between branch and tag.

A branch moves as new commits are added. A tag usually stays fixed at one commit.

3.21 Remote

What It Means

A remote is a reference to another copy of the repository, usually hosted on GitHub.

Why It Matters

Remotes enable collaboration.

Example

```
git remote -v
git remote add origin https://github.com/user/project.git
```

Common Interview Angle

`origin` is just the default remote name, not a special GitHub-only concept.

3.22 GitHub CLI Commands

What It Means

GitHub CLI, called `gh`, is a command-line tool for interacting with GitHub features such as repositories, pull requests, issues, workflows, and releases.

Why It Matters

It lets developers manage GitHub without leaving the terminal.

Common Commands

```
gh auth login
gh repo clone owner/repo
gh repo create
gh pr create
gh pr list
gh pr view
gh pr checkout 123
gh pr merge
gh issue create
gh issue list
gh workflow list
gh run list
gh run watch
gh release create v1.0.0
```

Common Interview Angle

Interviewers may not deeply test `gh`, but knowing it shows practical GitHub workflow awareness.

4. Real-World Example

Imagine a backend team building an e-commerce API.

Scenario

The production branch is `main`. Developers create separate branches:

```
main
|-- feature/user-login
|-- feature/payment
|-- bugfix/cart-total
```

Workflow

```
git clone https://github.com/company/shop-api.git
cd shop-api
git switch -c feature/user-login
```

The developer edits code:

```
git add src/auth.js
git commit -m "Add login endpoint"
git push -u origin feature/user-login
```

Then they create a pull request:

```
gh pr create --base main --head feature/user-login --title "Add login endpoint" --
body "Adds login API with validation"
```

CI runs tests. Reviewers approve. The PR is merged:

```
gh pr merge --squash
```

If another developer changed the same file, a conflict may occur. The developer resolves it, tests locally, pushes the fix, and the pull request updates automatically.

5. Diagrams / Mental Models

Git Areas

```
+-----+      git add      +-----+
| Working Directory | -----> | Staging Area |
+-----+                  +-----+
      ^                      |
      |                      | git commit
      | git restore          v
+-----+      git push      +-----+
| Remote Repository | <----- | Local History |
```

```

+-----+
|
| git fetch / git pull
v
Local Repository Updates

```

Branch Mental Model

```

A -- B -- C  main
      \
      D -- E  feature-login

```

`main` and `feature-login` are pointers to commits.

Merge Mental Model

Before merge:

```

A -- B -- C  main
      \
      D -- E  feature

```

After merge:

```

A -- B -- C ----- M  main
      \             /
      D -- E  ---

```

`M` is a merge commit.

Rebase Mental Model

Before rebase:

```

A -- B -- C  main
      \
      D -- E  feature

```

After rebase:

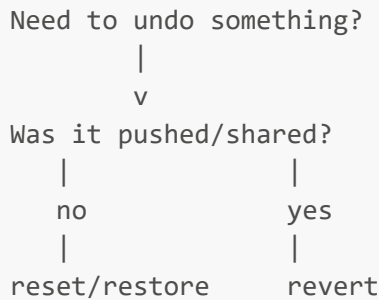
```

A -- B -- C -- D' -- E'  feature

```

The feature commits are replayed on top of `main`.

Undo Decision Flow



6. Common Interview Questions

Q1. What is Git?

Git is a distributed version control system that tracks changes in files using commits. It allows developers to collaborate, branch, merge, undo changes, and maintain project history.

Key points interviewer expects:

- Distributed VCS
- Tracks snapshots
- Supports collaboration
- Uses commits and branches

Common mistakes:

- Saying Git and GitHub are the same
- Saying Git only stores file differences
- Ignoring branches and commits

Q2. What is the difference between Git and GitHub?

Git is the version control tool. GitHub is a cloud platform that hosts Git repositories and provides pull requests, issues, reviews, Actions, and collaboration tools.

Key points interviewer expects:

- Git is local tool
- GitHub is remote hosting and collaboration platform
- Git can work without GitHub

Common mistakes:

- Treating GitHub as required for Git
- Saying GitHub stores code but Git does not

Q3. What happens when you run `git clone`?

`git clone` copies a remote repository to your local system. It downloads files, commit history, branches, tags, and sets up a default remote called `origin`.

Key points interviewer expects:

- Downloads repository history
- Creates working directory
- Sets remote tracking

Common mistakes:

- Saying it only downloads current files
- Confusing clone with pull

Q4. What is the staging area?

The staging area is an intermediate area where selected changes are prepared before committing. It allows developers to build clean, focused commits.

Key points interviewer expects:

- Between working directory and commit history
- Filled using `git add`
- Committed using `git commit`

Common mistakes:

- Thinking `git add` uploads files
- Thinking staging is the same as committing

Q5. What is the difference between `git pull` and `git fetch`?

`git fetch` downloads remote changes but does not integrate them into your branch. `git pull` downloads and integrates changes using merge or rebase.

Key points interviewer expects:

- `pull = fetch + merge` by default
- `fetch` is safer for inspection

Common mistakes:

- Saying both are identical
- Forgetting that pull changes the current branch

Q6. What is a branch in Git?

A branch is a lightweight movable pointer to a commit. It allows separate lines of development, such as features, bug fixes, or experiments.

Key points interviewer expects:

- Pointer to commit

- Lightweight
- Enables parallel development

Common mistakes:

- Saying a branch is a full copy of the project
- Thinking branches are expensive

Q7. What is the difference between merge and rebase?

Merge combines histories and may create a merge commit. Rebase moves or replays commits onto a new base, creating a linear history but rewriting commit hashes.

Key points interviewer expects:

- Merge preserves history
- Rebase rewrites history
- Rebase gives linear history
- Avoid rebasing public shared branches

Common mistakes:

- Saying rebase is always better
- Forgetting rebase creates new commits

Q8. What is a merge conflict?

A merge conflict occurs when Git cannot automatically combine changes, usually because two branches edited the same lines or one branch deleted a file another branch modified.

Key points interviewer expects:

- Happens during merge, rebase, cherry-pick, or pull
- Must be manually resolved
- Use `git status` and conflict markers

Common mistakes:

- Thinking conflicts mean Git is broken
- Keeping conflict markers in final code

Q9. How do you resolve a conflict?

Run `git status`, open conflicted files, inspect conflict markers, edit the file to the correct final version, remove markers, run tests, stage the file, and continue the merge or rebase.

Key points interviewer expects:

- Understand markers
- Manually choose correct code
- Stage after resolving
- Continue operation

Common mistakes:

- Accepting one side blindly
- Not running tests
- Forgetting `git rebase --continue`

Q10. What is the difference between reset and revert?

`git reset` moves the branch pointer and can rewrite history. `git revert` creates a new commit that undoes another commit without rewriting history.

Key points interviewer expects:

- Reset is common for local cleanup
- Revert is safer for shared history
- Hard reset can delete local changes

Common mistakes:

- Using reset on shared branches carelessly
- Thinking revert deletes the old commit

Q11. What does `git stash` do?

`git stash` temporarily saves uncommitted changes and cleans the working directory. Later, you can restore those changes using `git stash apply` or `git stash pop`.

Key points interviewer expects:

- Temporary save
- Useful before switching branches
- `pop` removes stash, `apply` keeps it

Common mistakes:

- Using stash as permanent storage
- Forgetting stashes are local

Q12. What is cherry-picking?

Cherry-picking applies a specific commit from one branch onto another branch without merging the entire branch.

Key points interviewer expects:

- Selective commit application
- Useful for hotfixes
- Can cause conflicts

Common mistakes:

- Thinking cherry-pick moves the commit
- Using it when a normal merge is better

Q13. What is HEAD in Git?

HEAD is a reference to the current checked-out commit or branch. Usually, it points to the latest commit on the current branch.

Key points interviewer expects:

- Current position
- Often points to branch
- Detached HEAD means HEAD points directly to a commit

Common mistakes:

- Thinking HEAD always means latest commit in the repo
- Not understanding detached HEAD

Q14. What is a pull request?

A pull request is a GitHub feature used to propose changes from one branch into another. It supports code review, CI checks, comments, and controlled merging.

Key points interviewer expects:

- Collaboration workflow
- Review before merge
- Usually branch to main/develop

Common mistakes:

- Saying PR is a core Git command
- Confusing PR with `git pull`

Q15. What is **origin**?

origin is the default name Git gives to the remote repository when you clone a project. It is a convention, not a keyword required by Git.

Key points interviewer expects:

- Remote alias
- Created during clone
- Can be renamed or changed

Common mistakes:

- Thinking origin always means GitHub
- Thinking origin is the local branch

7. Deep-Dive Questions

Q1. Why does rebase rewrite history?

Rebase takes commits from one branch and reapplies them onto another base commit. Since each Git commit depends on its parent commit, changing the parent changes the commit hash. Therefore, rebased commits are new commits with new hashes.

Q2. Why is `git reset --hard` dangerous?

`git reset --hard` moves the branch pointer, resets the staging area, and overwrites the working directory. Any uncommitted local changes can be lost. If used on shared history, it may also remove commits that teammates expect to exist.

Q3. What is a detached HEAD state?

A detached HEAD state occurs when `HEAD` points directly to a commit instead of a branch. You can inspect or experiment, but new commits may be hard to find unless you create a branch.

Example:

```
git checkout abc123
git switch -c experiment
```

Q4. How does Git identify commits?

Git identifies commits using SHA hashes based on commit content, metadata, parent commit, author information, timestamp, and tree data. This makes commits content-addressed and tamper-evident.

Q5. What happens internally during a merge?

Git finds the common ancestor of the two branches, compares changes from both sides, and tries to combine them. If changes do not overlap, Git merges automatically. If changes overlap in incompatible ways, Git reports conflicts.

8. Comparison Tables

Git vs GitHub

Feature	Git	GitHub
Type	Version control system	Hosting and collaboration platform
Works offline	Yes	No
Main purpose	Track history	Share, review, automate
Examples	<code>commit</code> , <code>branch</code> , <code>merge</code>	PRs, issues, Actions
Required for the other?	Git does not need GitHub	GitHub uses Git repositories

Clone vs Pull vs Fetch

Command	Purpose	When Used	Changes Working Branch?
<code>git clone</code>	Copy remote repo locally	First-time setup	Creates new local repo
<code>git fetch</code>	Download remote updates	Inspect before integration	No
<code>git pull</code>	Download and integrate updates	Update current branch	Yes

Add vs Commit vs Push

Command	Scope	Effect
<code>git add</code>	Staging area	Selects changes for next commit
<code>git commit</code>	Local repository	Saves staged changes locally
<code>git push</code>	Remote repository	Uploads local commits

Merge vs Rebase

Feature	Merge	Rebase
History style	Preserves branch history	Creates linear history
Commit hashes	Existing commits unchanged	Replayed commits get new hashes
Merge commit	Often yes	No
Safe for shared branch	Usually yes	Risky if rewriting public commits
Best use	Combining completed branches	Cleaning local feature branch history

Reset vs Revert

Feature	Reset	Revert
Rewrites history	Yes	No
Creates new commit	No	Yes
Safe on shared branch	Usually no	Yes
Common use	Local cleanup	Undo pushed commit
Risk level	High with <code>--hard</code>	Lower

Stash Apply vs Stash Pop

Command	Applies Changes	Removes Stash Entry	Use Case
<code>git stash apply</code>	Yes	No	Reuse stash later

Command	Applies Changes	Removes Stash Entry	Use Case
<code>git stash pop</code>	Yes	Yes	Restore and delete stash

Branch vs Tag

Feature	Branch	Tag
Moves with new commits	Yes	Usually no
Purpose	Active development	Mark release/version
Example	<code>feature-login</code>	<code>v1.0.0</code>
Can be pushed	Yes	Yes

Checkout vs Switch vs Restore

Command	Main Purpose
<code>git checkout</code>	Older command for switching branches and restoring files
<code>git switch</code>	Modern command for switching branches
<code>git restore</code>	Modern command for restoring file changes

9. Common Mistakes

- Confusing Git with GitHub.
- Thinking `git add` uploads files to GitHub.
- Making huge commits with unrelated changes.
- Pulling without checking local status.
- Using `git reset --hard` without understanding data loss.
- Rebasing shared branches that teammates are using.
- Resolving conflicts by blindly accepting one side.
- Forgetting to run tests after conflict resolution.
- Forgetting to push tags separately.
- Thinking a branch is a full copy of the repository.
- Using `git stash` as long-term storage.
- Not reading `git status` carefully.
- Using force push without understanding its effect.

10. Edge Cases / Special Cases

Detached HEAD

You are not on a branch. New commits can be lost from easy view unless you create a branch.

```
git switch -c save-my-work
```

Fast-Forward Merge

If the target branch has not diverged, Git can simply move the branch pointer forward.

```
Before:
A -- B  main
      \
       C -- D  feature

After fast-forward:
A -- B -- C -- D  main
```

Non-Fast-Forward Push Rejection

Your push is rejected when the remote has commits you do not have.

Fix usually:

```
git pull --rebase origin main
git push
```

Force Push

`git push --force` overwrites remote history. Prefer:

```
git push --force-with-lease
```

`--force-with-lease` checks that the remote has not changed unexpectedly.

Empty Commit

You can create a commit without file changes:

```
git commit --allow-empty -m "Trigger deployment"
```

Useful for CI/CD triggers.

Reverting a Merge Commit

Reverting a merge commit needs a mainline parent:

```
git revert -m 1 <merge-commit>
```

This is advanced and should be done carefully.

Stashing Untracked Files

By default, `git stash` may not include untracked files.

```
git stash -u
```

Ignored Files

Files listed in `.gitignore` are ignored only if they are not already tracked. If a file is already tracked, adding it to `.gitignore` does not automatically remove it from Git history.

```
git rm --cached file.log
```

11. How to Explain in Interview

Git is a distributed version control system that tracks project changes through commits. I use branches to develop features independently, commits to save logical checkpoints, and push or pull to collaborate through remotes like GitHub. For team workflows, I usually create a feature branch, push it, open a pull request, resolve review comments or conflicts, and merge it into the main branch after tests pass. For undoing changes, I use reset for local unpublished cleanup and revert for already shared commits because revert preserves history.

12. Quick Revision Notes

Key Definitions

- Repository: A Git-tracked project.
- Commit: A snapshot of staged changes.
- Branch: A pointer to a commit.
- Remote: Another copy of the repository, usually hosted online.
- HEAD: Current checked-out commit or branch.
- Staging area: Place where changes are prepared before commit.
- Merge: Combine branch histories.
- Rebase: Replay commits onto a new base.
- Stash: Temporarily save uncommitted work.
- Revert: Create a new commit that undoes an old commit.
- Reset: Move branch pointer and optionally change staging/working directory.

Important Points

- `git pull` = `git fetch` + `git merge` by default.
- `git add` stages changes; it does not upload them.
- `git commit` saves locally; `git push` uploads remotely.
- Branches are lightweight pointers.
- Rebase rewrites history.

- Revert is safer than reset on shared branches.
- `git status` is the first debugging command.
- Resolve conflicts manually, then stage files.

Common Comparisons

- Git vs GitHub
- Fetch vs Pull
- Merge vs Rebase
- Reset vs Revert
- Stash Apply vs Stash Pop
- Branch vs Tag

Must-Remember Facts

- Do not rebase public branches unless the team agrees.
- Avoid `git reset --hard` unless you are sure.
- Prefer `--force-with-lease` over `--force`.
- Pull requests are GitHub features, not core Git commands.
- Tags are commonly used for releases.
- `.gitignore` does not untrack files already committed.

Interview Traps

- "Git and GitHub are same" is wrong.
- "Branch is a copy" is wrong.
- "Pull and fetch are same" is wrong.
- "Revert deletes commit" is wrong.
- "Reset is always safe" is wrong.

13. Practice Tasks

Task 1: Basic Local Workflow

```
mkdir git-practice
cd git-practice
git init
echo "hello" > app.txt
git add app.txt
git commit -m "Add app file"
git log --oneline
```

Goal: Understand init, add, commit, and log.

Task 2: Branch and Merge

```
git switch -c feature-message
echo "new message" >> app.txt
git add app.txt
git commit -m "Add new message"
git switch main
git merge feature-message
```

Goal: Understand branch development and merging.

Task 3: Create and Resolve a Conflict

```
git switch -c branch-a
echo "Version A" > conflict.txt
git add conflict.txt
git commit -m "Add version A"

git switch main
git switch -c branch-b
echo "Version B" > conflict.txt
git add conflict.txt
git commit -m "Add version B"

git switch branch-a
git merge branch-b
```

Goal: Practice conflict markers and manual resolution.

Task 4: Practice Stash

```
echo "temporary work" >> app.txt
git stash
git status
git stash pop
```

Goal: Understand temporary saving.

Task 5: Reset Modes

```
echo "reset test" >> app.txt
git add app.txt
git commit -m "Test reset"
git reset --soft HEAD~1
git status
```

Repeat with `--mixed` and `--hard` in a practice repository.

Goal: Understand the difference between reset modes.

Task 6: Revert a Commit

```
echo "bad change" >> app.txt
git add app.txt
git commit -m "Add bad change"
git revert HEAD
```

Goal: Learn safe undo for shared history.

Task 7: Cherry-Pick

```
git switch -c hotfix
echo "hotfix" >> app.txt
git add app.txt
git commit -m "Add hotfix"
git switch main
git cherry-pick hotfix
```

Goal: Apply one commit from another branch.

Task 8: GitHub CLI Pull Request Flow

```
gh auth login
gh repo create git-practice-remote --public --source=. --remote=origin --push
git switch -c feature-readme
echo "# Git Practice" > README.md
git add README.md
git commit -m "Add README"
git push -u origin feature-readme
gh pr create --base main --head feature-readme --title "Add README" --body "Adds
project README"
```

Goal: Practice GitHub CLI workflow.

Task 9: Inspect History

```
git log --oneline --graph --decorate --all
git diff HEAD~1..HEAD
git show HEAD
```

Goal: Learn how to inspect commits and changes.

Task 10: Tag a Release

```
git tag v1.0.0
git tag
git push origin v1.0.0
```

Goal: Understand release tagging.

14. Final Cheat Sheet

Core Definition

Git is a distributed version control system that tracks code history using commits and enables collaboration through branches, merges, and remotes.

Why It Matters

Git helps teams safely develop features, review code, undo mistakes, manage releases, and collaborate on real-world software projects.

Most Asked Questions

Question	Short Answer
Git vs GitHub?	Git is VCS; GitHub hosts Git repos and collaboration tools.
Fetch vs Pull?	Fetch downloads only; pull downloads and integrates.
Merge vs Rebase?	Merge preserves history; rebase rewrites commits for linear history.
Reset vs Revert?	Reset rewrites/moves history; revert creates undo commit.
What is staging?	Area where changes are prepared before commit.
What is branch?	Lightweight pointer to a commit.
What is conflict?	Git cannot automatically combine changes.

Common Commands

```
git init
git clone <url>
git status
git add <file>
git commit -m "message"
git push origin <branch>
git pull origin <branch>
git fetch origin
git branch
```

```
git switch -c <branch>
git merge <branch>
git rebase <branch>
git stash
git reset --soft HEAD~1
git reset --mixed HEAD~1
git reset --hard HEAD~1
git revert <commit>
git cherry-pick <commit>
git log --oneline --graph --decorate --all
git diff
git restore <file>
git tag v1.0.0
```

GitHub CLI Commands

```
gh auth login
gh repo clone owner/repo
gh repo create
gh pr create
gh pr list
gh pr view
gh pr checkout <number>
gh pr merge
gh issue create
gh issue list
gh workflow list
gh run list
gh run watch
gh release create v1.0.0
```

Common Comparisons

Comparison	Must Remember
Git vs GitHub	Tool vs platform
Add vs Commit vs Push	Stage vs save locally vs upload
Fetch vs Pull	Download vs download plus integrate
Merge vs Rebase	Preserve history vs rewrite for linear history
Reset vs Revert	Local history rewrite vs safe undo commit
Branch vs Tag	Moving pointer vs fixed release marker

One-Line Interview Answer

Git is a distributed version control system that lets developers track code changes, work on branches, collaborate through remotes like GitHub, and safely manage history using commands like commit, merge,

rebase, reset, and revert.